07. SGX vs TDX 차이점 및 특화 처리

SGX와 TDX 개요

SGX (Software Guard Extensions)

- **레벨**: 프로세서 레벨 보안 기술
- **격리 단위**: Enclave (애플리케이션 내 보안 영역)
- **보호 대상**: 특정 코드 및 데이터
- **위협 모델**: 특권 소프트웨어(OS, 하이퍼바이저)로부터 보호

TDX (Trust Domain Extensions)

- **레벨**: 가상화 레벨 보안 기술
- **격리 단위**: Trust Domain (전체 가상 머신)
- **보호 대상**: 전체 게스트 OS 및 애플리케이션
- **위협 모델**: 호스트 OS, 하이퍼바이저, 다른 TD로부터 보호

TEE 타입 식별

상수 정의

```rust
pub const TEE_TYPE_SGX: u32 = 0x00000000;

pub const TEE_TYPE_TDX: u32 = 0x00000081;
```

Header에서 TEE 타입 확인

```
impl Header {
pub fn is_sgx(&self) -> bool {
self.tee_type == TEE_TYPE_SGX
}
}

// 사용 예시
if quote.header.is_sgx() {
// SGX 전용 처리
} else if quote.header.tee_type == TEE_TYPE_TDX {
// TDX 전용 처리
}
```

Quote 버전별 지원 매트릭스

Quote Version 3

- **지원 TEE**: SGX만 지원
- **Report 타입**: `EnclaveReport` 만 사용
- **특징**: 초기 SGX DCAP 구현

```
match header.version {
```

```
3 => {
if  header.tee_type != TEE_TYPE_SGX {
return Err(scale::Error::from("invalid tee type"));
}
report = Report::SgxEnclave(EnclaveReport::decode(input)?);
}
// ...
}
```

Quote Version 4

- **지원 TEE**: SGX, TDX 모두 지원
- **Report 타입**: TEE 타입에 따라 분기
- **특징**: TDX 지원 추가

```
4 => {
match  header.tee_type {
TEE_TYPE_SGX => {
report = Report::SgxEnclave(EnclaveReport::decode(input)?);
}
TEE_TYPE_TDX => {
report = Report::TD10(TDReport10::decode(input)?);
}
_ => return Err(scale::Error::from("Invalid TEE type")),
```

```
}

}
```

Quote Version 5

- **지원 TEE**: SGX, TDX 모두 지원
- **Report 타입**: Body 구조를 통해 명시적 지정
- **특징**: 확장된 TDX Report 지원 (TD15)

```
5 => {

let  body = Body::decode(input)?;

match  body.body_type {

BODY_SGX_ENCLAVE_REPORT_TYPE => {

report = Report::SgxEnclave(EnclaveReport::decode(input)?);

}

BODY_TD_REPORT10_TYPE => {

report = Report::TD10(TDReport10::decode(input)?);

}

BODY_TD_REPORT15_TYPE => {

report = Report::TD15(TDReport15::decode(input)?);

}

_ => return Err(scale::Error::from("Unsupported body type")),

}

}
```

Report 구조 차이점

SGX EnclaveReport (384바이트)

```rust
pub struct EnclaveReport {
pub cpu_svn: [u8; 16], // CPU 보안 버전
pub misc_select: u32, // 기타 선택 필드
pub reserved1: [u8; 28], // 예약 필드
pub attributes: [u8; 16], // Enclave 속성
pub mr_enclave: [u8; 32], // Enclave 측정값 (코드 해시)
pub reserved2: [u8; 32], // 예약 필드
pub mr_signer: [u8; 32], // 서명자 측정값 (공개키 해시)
pub reserved3: [u8; 96], // 예약 필드
pub isv_prod_id: u16, // ISV 제품 ID
pub isv_svn: u16, // ISV 보안 버전
pub reserved4: [u8; 60], // 예약 필드
pub report_data: [u8; 64], // 사용자 데이터
}
```

TDX TDReport10 (584바이트)

```rust
pub struct TDReport10 {
pub tee_tcb_svn: [u8; 16], // TEE TCB 보안 버전
pub mr_seam: [u8; 48], // SEAM 모듈 측정값
```

```
pub mr_signer_seam: [u8; 48], // SEAM 서명자 측정값
pub seam_attributes: [u8; 8], // SEAM 속성
pub td_attributes: [u8; 8], // TD 속성
pub xfam: [u8; 8], // 확장 기능 가용성 마스크
pub mr_td: [u8; 48], // TD 측정값 (전체 TD 해시)
pub mr_config_id: [u8; 48], // 구성 ID 측정값
pub mr_owner: [u8; 48], // 소유자 측정값
pub mr_owner_config: [u8; 48], // 소유자 구성 측정값
pub rt_mr0: [u8; 48], // 런타임 측정값 0
pub rt_mr1: [u8; 48], // 런타임 측정값 1
pub rt_mr2: [u8; 48], // 런타임 측정값 2
pub rt_mr3: [u8; 48], // 런타임 측정값 3
pub report_data: [u8; 64], // 사용자 데이터
}
```

TDX TDReport15 (648바이트)

```
pub struct TDReport15 {
pub base: TDReport10, // 기본 TDReport10 포함
pub tee_tcb_svn2: [u8; 16], // 추가 TEE TCB 보안 버전
pub mr_service_td: [u8; 48], // 서비스 TD 측정값
}
```

측정값(Measurement) 차이점

SGX 측정값

- **MR_ENCLAVE**: Enclave 내 코드와 초기 데이터의 해시 (32바이트)
- **MR_SIGNER**: Enclave 서명자의 공개키 해시 (32바이트)
- **범위**: 특정 Enclave 애플리케이션

TDX 측정값

- **MR_TD**: 전체 Trust Domain의 측정값 (48바이트)
- **MR_CONFIG_ID**: TD 구성 식별자 (48바이트)
- **MR_OWNER**: TD 소유자 식별자 (48바이트)
- **RT_MRx**: 런타임 측정값들 (각 48바이트)
- **범위**: 전체 가상 머신 환경

속성(Attributes) 검증 차이

SGX 속성 검증

```
fn validate_sgx(report: &EnclaveReport) -> Result<()> {
// DEBUG 비트 확인 (비트 1)
let is_debug = report.attributes[0] & 0x02 != 0;
if is_debug {
bail!("Debug mode is enabled");
}
```

```
Ok(())
}
```

TDX 속성 검증

```
fn validate_td10(report: &TDReport10) -> Result<()> {
let td_attrs = TDAttributes::parse(report.td_attributes)?;
// TUD (TD Under Debug) 플래그 확인
if td_attrs.tud != 0 {
bail!("Debug mode is enabled");
}
// 예약 비트 확인
if td_attrs.sec.reserved_lower != 0
|| td_attrs.sec.reserved_bit29
|| td_attrs.other.reserved != 0
{
bail!("Reserved bits in TD attributes are set");
}
// 보안 속성 확인
if !td_attrs.sec.sept_ve_disable {
bail!("SEPT_VE_DISABLE is not enabled");
}
if td_attrs.sec.pks {
```

```
bail!("PKS is enabled");

}

if td_attrs.sec.kl {

bail!("KL is enabled");

}

Ok(())

}
```

TCB 구성 요소 차이

SGX TCB 구성 요소

```
// SGX만 사용하는 구성 요소

let sgx_components = tcb_level

.tcb

.sgx_components

.iter()

.map(|c| c.svn)

.collect::<Vec<_>>();


// CPU SVN과 비교 (16바이트)

if cpu_svn[..] < sgx_components[..] {

continue;
```

```
}
```

TDX 추가 TCB 구성 요소

```
// TDX의 경우 추가 구성 요소 확인
if quote.header.tee_type == TEE_TYPE_TDX {
let td_report = quote.report.as_td10()?;
// TDX 전용 구성 요소
let tdx_components = tcb_level
.tcb
.tdx_components
.iter()
.map(|c| c.svn)
.collect::<Vec<_>>();
if tdx_components.is_empty() {
bail!("No TDX components in the TCB info");
}
// TEE TCB SVN과 비교 (16바이트)
if td_report.tee_tcb_svn[..] < tdx_components[..] {
continue;
}
}
```

Collateral 수집 차이점

엔드포인트 URL 차이

```
impl PcsEndpoints {
fn new(base_url: &str, for_sgx: bool, fmspc: String, ca: &'static str) -> Self
let tee = if for_sgx { "sgx" } else { "tdx" }; // URL 경로 차이
// ...
}
// TCB 정보 엔드포인트
fn url_tcb(&self) -> String {
// SGX: /sgx/certification/v4/tcb?fmspc=...
// TDX: /tdx/certification/v4/tcb?fmspc=...
self.mk_url(self.tee, &format!("tcb?fmspc={}", self.fmspc))
}
// QE Identity 엔드포인트
fn url_qe_identity(&self) -> String {
// SGX: /sgx/certification/v4/qe/identity?update=standard
// TDX: /tdx/certification/v4/qe/identity?update=standard
self.mk_url(self.tee, "qe/identity?update=standard")
}
}
```

TCB 정보 헤더 차이

```
// SGX의 경우
tcb_info_issuer_chain = get_header(&response, "SGX-TCB-Info-Issuer-Chain")?;


// TDX의 경우 (fallback 지원)
tcb_info_issuer_chain = get_header(&response, "SGX-TCB-Info-Issuer-Chain")
.or(get_header(&response, "TCB-Info-Issuer-Chain"))?;
```

TCB 정보 검증 차이

TDX TCB 정보 특별 검증

```
// TDX Quote의 경우 TCB 정보 호환성 확인
if  quote.header.tee_type == TEE_TYPE_TDX && (tcb_info.version < 3 || tcb_info.i
bail!("TDX quote with non-TDX TCB info in the collateral");
}
```

TCB ID 확인

- **SGX**: `tcb_info.id == "SGX"`
- **TDX**: `tcb_info.id == "TDX"`

Report 접근자 함수

타입별 Report 접근

```
impl Report {
pub fn is_sgx(&self) -> bool {
matches!(self, Report::SgxEnclave(_))
}

pub fn as_td10(&self) -> Option<&TDReport10> {
match self {
Report::TD10(report) => Some(report),
Report::TD15(report) => Some(&report.base), // TD15는 TD10 포함
_ => None,
}
}

pub fn as_td15(&self) -> Option<&TDReport15> {
match self {
Report::TD15(report) => Some(report),
_ => None,
}
}

pub fn as_sgx(&self) -> Option<&EnclaveReport> {
```

```
match self {
Report::SgxEnclave(report) => Some(report),
_ => None,
}
}
}
```

서명된 데이터 길이 계산

TEE별 길이 차이

```
impl Quote {
pub fn signed_length(&self) -> usize {
let mut len = match self.report {
Report::SgxEnclave(_) => HEADER_BYTE_LEN + ENCLAVE_REPORT_BYTE_LEN, // 48 + 384
Report::TD10(_) => HEADER_BYTE_LEN + TD_REPORT10_BYTE_LEN, // 48 + 584 = 632
Report::TD15(_) => HEADER_BYTE_LEN + TD_REPORT15_BYTE_LEN, // 48 + 648 = 696
};
if self.header.version == 5 {
len += BODY_BYTE_SIZE; // Quote v5는 6바이트 Body 추가
}
len
}
```

```
}
```

실제 사용 예시

TEE 타입별 처리 분기

```
match  quote.header.tee_type {
TEE_TYPE_SGX => {
let  sgx_report = quote.report.as_sgx().unwrap();
println!("SGX Enclave MR_ENCLAVE: {:?}", hex::encode(sgx_report.mr_enclave));
println!("SGX Enclave MR_SIGNER: {:?}", hex::encode(sgx_report.mr_signer));
}
TEE_TYPE_TDX => {
let  td_report = quote.report.as_td10().unwrap();
println!("TDX MR_TD: {:?}", hex::encode(td_report.mr_td));
println!("TDX MR_CONFIG_ID: {:?}", hex::encode(td_report.mr_config_id));
}
_ => {
bail!("Unsupported TEE type");
}
}
```

이러한 차이점들을 통해 SGX와 TDX 각각의 특성에 맞는 적절한 검증과 처리가 이루어집니다.